

1)Понятие системы счисления

Система счисления - это совокупность правил и приемов записи чисел с помощью набора цифровых знаков. Количество цифр, необходимых для записи числа в системе, называют основанием системы счисления. Основание системы записывается в справа числа в нижнем индексе: ; ; и т. д.

Различают два типа систем счисления:

позиционные, когда значение каждой цифры числа определяется ее позицией в записи числа;

непозиционные, когда значение цифры в числе не зависит от ее места в записи числа.

Примером непозиционной системы счисления является римская: числа IX, IV, XV и т.д. Примером позиционной системы счисления является десятичная система, используемая повседневно.

2)Перевод чисел из одной системы счисления в другую.

1. Для перевода двоичного числа в десятичное необходимо его записать в виде многочлена, состоящего из произведений цифр числа и соответствующей степени числа 2, и вычислить по правилам десятичной арифметики:

$$X_2 = A_n \cdot 2^{n-1} + A_{n-1} \cdot 2^{n-2} + A_{n-2} \cdot 2^{n-3} + \dots + A_2 \cdot 2^1 + A_1 \cdot 2^0$$

2. Для перевода восьмеричного числа в десятичное необходимо его записать в виде многочлена, состоящего из произведений цифр числа и соответствующей степени числа 8, и вычислить по правилам десятичной арифметики:

$$X_8 = A_n \cdot 8^{n-1} + A_{n-1} \cdot 8^{n-2} + A_{n-2} \cdot 8^{n-3} + \dots + A_2 \cdot 8^1 + A_1 \cdot 8^0$$

3. Для перевода шестнадцатеричного числа в десятичное необходимо его записать в виде многочлена, состоящего из произведений цифр числа и соответствующей степени числа 16, и вычислить по правилам десятичной арифметики:

$$X_{16} = A_n \cdot 16^{n-1} + A_{n-1} \cdot 16^{n-2} + A_{n-2} \cdot 16^{n-3} + \dots + A_2 \cdot 16^1 + A_1 \cdot 16^0$$

4. Для перевода десятичного числа в двоичную систему его необходимо последовательно делить на 2 до тех пор, пока не останется остаток, меньший или равный 1. Число в двоичной системе записывается как последовательность последнего результата деления и остатков от деления в обратном порядке.

5. Для перевода десятичного числа в восьмеричную систему его необходимо последовательно делить на 8 до тех пор, пока не останется остаток, меньший или равный 7. Число в восьмеричной системе записывается как последовательность цифр последнего результата деления и остатков от деления в обратном порядке.

6. Для перевода десятичного числа в шестнадцатеричную систему его необходимо последовательно делить на 16 до тех пор, пока не останется остаток, меньший или равный 15. Число в шестнадцатеричной системе записывается как последовательность цифр последнего результата деления и остатков от деления в обратном порядке.

7. Чтобы перевести число из двоичной системы в восьмеричную, его нужно разбить на триады (тройки цифр), начиная с младшего разряда, в случае необходимости дополнив старшую триаду нулями, и каждую триаду заменить соответствующей восьмеричной цифрой (табл. 3).

8. Чтобы перевести число из двоичной системы в шестнадцатеричную, его нужно разбить на тетрады (четверки цифр), начиная с младшего разряда, в случае необходимости дополнив старшую тетраду нулями, и каждую тетраду заменить соответствующей восьмеричной цифрой (табл. 3).

9. Для перевода восьмеричного числа в двоичное необходимо каждую цифру заменить эквивалентной ей двоичной триадой.

10. Для перевода шестнадцатеричного числа в двоичное необходимо каждую цифру заменить эквивалентной ей двоичной тетрадой.

11. При переходе из восьмеричной системы счисления в шестнадцатеричную и обратно, необходим промежуточный перевод чисел в двоичную систему.

3)Представление чисел с фиксированной и плавающей запятой в ЭВМ.

I. В режиме с фиксированной точкой	II. В режиме с плавающей точкой	I. В режиме с фиксированной точкой	II. В режиме с плавающей точкой
Знак числа – 1 разряд,	Знак числа (мантиссы) – 1 разряд,	$+ X_{\text{м.к.}} = +999.99999$	$+ X_{\text{м.к.}} = +0.99999 \cdot 10^{+99}$
Целая часть – 3 разряда,	Мантисса – 5 разрядов,		
Дробная часть – 5 разрядов,	Знак порядка – 1 разряд,		
Итого – 9 разрядов.	Порядок числа – 2 разряда,	$+ X_{\text{м.к.}} = +000.00001$	$+ X_{\text{м.к.}} = +0.10000 \cdot 10^{-99}$
	Итого – 9 разрядов.	000.00000	0.00000
		$- X_{\text{м.к.}} = -000.00001$	$- X_{\text{м.к.}} = +0.10000 \cdot 10^{-99}$
			
			
		$- X_{\text{м.к.}} = -999.99999$	$- X_{\text{м.к.}} = -0.99999 \cdot 10^{+99}$

9	8	7	6	5	4	3	2	1
---	---	---	---	---	---	---	---	---

Знак Целая часть Дробная часть

9	8	7	6	5	4	3	2	1
---	---	---	---	---	---	---	---	---

Знак порядка ↑ Мантисса
Знак мантиссы

4)Форматы данных, прямой, обратный, дополнительный код.

Прямой код: Пусть X –правильная двоичная дробь (положительная или отрицательная). В частном случае X равно нулю.

Прямой код числа X называется число, обозначаемое символом $[X]_{пр}$, получаемое по следующей формуле:

$[X]_{пр} = (X, \text{ если } X \geq 0, 1-X, \text{ если } X < 0)$. Следовательно, прямой код двоичного числа совпадает по изображению с записью самого числа, при этом в знаковом разряде для положительных чисел записывается 0, а для отрицательных 1. Пример: $X = -0,1101$, $[X]_{пр} = [-0,1101] = 1 - (-0,1101) = 1,1101$

Обратный код: Пусть X – правильная двоичная дробь, в частности $X \leq 0$. Обратным кодом числа X называется число $[X]_{об} = (X, \text{ если } X \geq 0, 10 - 10^p + X, \text{ если } X < 0)$ где p – количество цифр дробной части числа X , 10 – означает два. Обратный код отрицательного двоичного числа получается следующим образом: в знаковый разряд записывается 1, а значения всех двоичных цифр дробной части инвертируются, т.е. заменяются на противоположные.

+Дополнительный код: $[X]_{доп} = (X, \text{ если } X \geq 0, 10 + X, \text{ если } X < 0)$, где 10 – означает два. Дополнительный код отрицательного двоичного числа получается следующим образом: в знаковый разряд записывается 1, значения всех двоичных разрядов дробной части инвертируются, а к младшему разряду полученного при этом числа прибавляется единица по правилам двоичной арифметики. Пример: $X = -0,1101$, $[X]_{доп} = [-0,1101] = 10 + (-0,1101) = 1,0011$. Для перевода из дополнительного кода отрицательного числа в прямой код необходимо из младшего разряда вычесть единицу, а затем все цифровые разряды инвертировать, оставив знаковый разряд без изменения. Модифицированный дополнительный код так же отличается от просто дополнительного двумя знаковыми разрядами.

5)Выполнение операции алгебраического сложения в ЭВМ.

При вычислении суммы двух чисел возможны два варианта: слагаемые имеют одинаковые знаки и слагаемые имеют различные знаки. В результате этого алгоритмы получения суммы для каждого из них различны.

Для операндов с одинаковыми знаками:

1. Сложить два числа.
2. Сумме присвоить знак одного из слагаемых.

Алгоритм получения алгебраической суммы:

1. Сравнить знаки слагаемых, и если они одинаковы, то выполнить сложение по первому алгоритму.
2. Если знаки слагаемых разные, то сравнить слагаемые по абсолютной величине.
3. Вычесть из большего меньшее.
4. Результату присвоить знак большего слагаемого.

Из этого следует, что первый алгоритм проще второго. Следовательно, желательно преобразовать отрицательные числа таким образом, чтобы операцию вычитания заменить операцией сложения, т.е. $S = A + (-B)$.

Для того, чтобы решить эту проблему, необходимо вводить специальные коды: прямой, обратный, дополнительный.

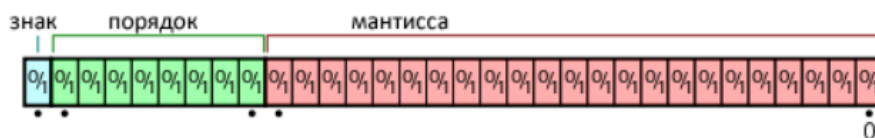
Способ построения этих кодов определяется функциями кодирования, которые должны обеспечить:

1. Запись алгебраического знака числа.
2. Представление отрицательных чисел при помощи вспомогательных, положительных, которые отличаются по изображению от положительных исходных чисел, т.е. области изображений положительных и отрицательных чисел не должны пересекаться.
3. Полную идентичность алгоритмов выполнения операций над числами различных знаков, и следовательно, полную идентичность необходимого при этом оборудования.

6) Арифметика чисел с плавающей запятой. Погрешности представления

Числа с плавающей запятой — один из возможных способов представления действительных чисел, который является компромиссом между точностью и диапазоном принимаемых значений.

Число с плавающей запятой состоит из набора отдельных разрядов, условно разделенных на знак, экспоненту порядок и мантиссу. Порядок и мантисса — целые числа, которые вместе со знаком дают представление числа с плавающей запятой в следующем виде:



7) Умножение двоичных чисел.

Операция умножения в двоичной системе счисления имеет одну характерную особенность. Так как очередная цифра множителя может быть только 0 или 1, то промежуточное (частичное) произведение равно либо множимому, либо 0. Таким образом, операция умножения в двоичной системе фактически не производится: промежуточные произведения сдвигаются влево на один разряд и складываются. Иначе говоря, операция умножения заменяется последовательным сложением.

8) Методы ускорения выполнения операции умножения.

Способы ускорения операции умножения подразделяются на логические и аппаратные.

Под аппаратными понимают такие способы, которые требуют для своей реализации введения дополнительного оборудования в основные арифметические цепи, благодаря чему достигается совмещение во времени отдельных составных частей процесса умножения.

Под логическими понимают такие способы ускорения, при реализации которых сохраняется без каких-либо изменений основная структура арифметических цепей умножителя, а ускорение достигается только за счет усложнения схемы управления.

Простейшим логическим способом является пропуск тактов суммирования в тех случаях, когда очередная цифра множителя равна нулю.

Другим способом является одновременное умножение на два разряда.

Для младшей пары разрядов при умножении с младших разрядов возможны следующие комбинации единиц и нулей в разрядах: 00; 01; 10; 11.

Для первой комбинации не производится ни сложение, ни вычитание, для второй - суммирование множимого, для третьей суммирование сдвинутого на один разряд влево множимого, для четвертой - одно вычитание множимого и одно сложение после сдвига множимого на 2 разряда.

Т.к. сложение после сдвига приходится на умножение на следующую пару разрядов, то, вместо того, чтобы его выполнять, добавляют единицу в следующую за данной парой разрядов.

Описанная процедура повторяется для всех пар разрядов множителя, а также для одной пары разрядов левее занятой, т.к. может оказаться необходимым добавить к ней единицу.

+Ускорение умножения достигается за счет того, что при комбинации 11 два сложения заменяются одним вычитанием, которое эквивалентно сложению благодаря использованию инверсных кодов, а добавление 1 в соседнюю старшую группу множителя производится одновременно с вычитанием множимого и не увеличивает время умножения.

9) Деление двоичных чисел в прямом коде

Деление осуществляется следующим образом.

Определяется знак частного путем сложения по модулю два знаков делимого и делителя. Затем производится собственно деление. При этом цифры частного определяются последовательно, разряд за разрядом, начиная со старшего разряда, путем вычитания, например, сдвинутого на один разряд вправо делителя из остатка, полученного от предыдущего вычитания. При определении первой цифры частного, за остаток принимается все делимое со знаком плюс. После каждого вычитания делитель сдвигается вправо на один разряд по отношению к делимому и образующимся остаткам. Если остаток от вычитания положительный или равен нулю, то в соответствующий разряд частного заносится 1. Если остаток отрицательный, то соответствующая цифра частного равна 0. Для того, чтобы получить следующую после нуля цифру частного, можно из последнего положительного остатка вычесть делитель, дополнительно сдвинутый на один разряд вправо. Однако в этом случае необходимо выполнить дополнительную операцию сложения для восстановления последнего положительного остатка.

10) Деление двоичных чисел в дополнительных кодах.

Делимое в $\bar{Q}$, делитель в R_2 , частное в P_1 . В начале каждого цикла сравнивается знак. разряд на $\bar{Q}$ и P_2 . Если они совпадают, то очередная цифра частного 1 и вследствие также вычисляется из $\bar{Q}$; Если знаки не совпадают, то 0 и «+» также, выполняется сдвиг в сторону старших разрядов $\bar{Q}$ и P_1 . Далее выполняется поправка. К результату «+» поправка вида 1, 00..01, если не совпадают.

11) Ускоренные методы операции деления.

Уменьшается количество операций.

Если 0,0xx операция не выполняется, в регистре частного наносится 0 и складывается.

1,1xx заносится 1 и «-»

+Алгоритм с анализом 1-го разряда после запятой.

Лог. операция	$\square = x, x$	Выполнение
Вычитание	0,0 0,1	Нет C:=0 вычитание
Сложение	1,1 1,0	Нет C:=1 сложение

Сдвиг выполняется в любом случае.

Метод без восстановления остатка позволяет в саму операцию включать проверку на возможность деления.

Ускоренный метод с анализом 2-х разр. После «,».

Разряды	B = 0,10...	B = 0,11...
0,00...	□ < B, А.О нет	□ < B, А.О нет
0,01...	□ < B, нет	□ < B, нет
0,10...	□ > < B, есть	□ < B, нет
0,11...	□ > B, есть	□ > < B, есть
1,...	□ > B, есть	□ > B, есть

1,... возможно переполнение, операция выполняется.

Данная таблица соответствует прямым кодам.

12) Извлечение корня из двоичных чисел.

Алгоритм извлечения квадратного корня. Необходимо выполнить n подряд циклов, где n – число разрядов после «,». Каждый цикл содержит 3 фазы:

Из суммы вычитается очередной результат извлечения кв-го корня с приписанной к младшему разряду пары 01. Если рез-т «-» очередная цифра 0, если «+» цифра 1.

В случае «-» рез-та выполняется восстановление текущего остатка.

Производится сдвиг, содержащий сумму, в сторону старшего разряда. Р в старшую сторону.

При работе с числами в формате с плавающей «,» необходимо учитывать:

+

если порядок х - четной степени, то порядок результата получается сдвигом в сторону младшего разряда. (делится на 2)

если порядок нечетный, то операнд приводится к четному порядку. Мантисса сдвигается в сторону младших разрядов.

13) Двоично-десятичные коды (D-коды), их разновидности, области применения.

Двоично-десятичный код — форма записи рациональных чисел, когда каждый десятичный разряд числа записывается в виде его четырёхбитного двоичного кода. Таким образом, каждая тетрада двоично-десятичного числа может принимать значения от 0000_2 (0_{10}) до 1001_2 (9_{10}).

Этот код удобен для выполнения машинных преобразований из десятичной системы в двоичную и обратно, а также для выполнения арифметических операций в D-кодах. Данный код аддитивен, то есть сумма представлений n цифр есть код их суммы. Знаки плюс и минус: + : 0000; - : 1001.

Код 8421 не является самоопределяющимся, то есть инверсия его двоичных цифр не даёт кода дополнения десятичной цифры до 9.

В связи с этим, в процессе выполнения арифметических действий над двоично-десятичными числами,

их коды преобразуются из системы 8421 либо в код с потетрадным избытком 3, либо в код с потетрадным избытком 6, являющимися и обеспечивающие лёгкое определение переноса из разряда в разряд, ввиду исключения лишних кодовых комбинаций (1010; 1011; 1100; 1101; 1110; 1111).

+Коды с потетрадным избытком 3 или 6 обеспечивают возможность применения достаточно простых в структурном отношении алгоритмов и устройств для выполнения арифметических операций в D-кодах. Вычитание из тетрады избытка, т.е. числа 6 (0110) можно заменить потетрадным прибавлением его дополнения 24, то есть числа 1010.

14) Особенности выполнения операции сложения в D-кодах.

Сложить два десятичных числа $X=+95$ и $Y=-7$.

Естественные двоично-десятичные коды чисел X и Y соответственно будут иметь вид $X=+1001\ 0101$ и $Y=-0111\ 1000$.

Образуем дополнение числа -78 до 102 с избытком 6 в каждой путём инверсии разрядов кода $0111\ 1000$ и прибавлением 1 в младший разряд младшей тетрады, тогда $1000\ 1000 = [Y]_{\text{доп(с избытком 6)}}$.

Теперь, опуская для простоты знаковые разряды, произведём сложение кода X с кодом $[Y]_{\text{доп}}$, помня, что для положительных чисел прямой и дополнительный коды совпадают

$1001\ 0101\ X + 1000\ 1000\ [Y]_{\text{доп(с избытком 6)}} \leftarrow 0001\ 1101\ [X+Y]_{\text{пр.}}$ (сумма в прямом коде с избытком в первой тетраде)

+ 0000 1010 коррекция

$0001\ 0111\ X+Y = 17$ (истинное значение суммы).

Если при сложении нет переносов из каких-либо тетрад, то после выполнения сложения они должны корректироваться путём вычитания из каждой тетрады числа 6, то есть прибавления кода 1010, являющегося дополнением 6 до 16.

+Наличие переноса из старшей тетрады при сложении операндов с различными знаками свидетельствует о том, что результат получился в прямом коде, а отсутствие переноса свидетельствует о том, что результат получился в дополнительном коде.

15) Получение дополнительного кода чисел в D-кодах.

Найти разность десятичных чисел $X=+67$ и $Y=-83$, т.е. выполнить $X - Y = 67 - 83 = 67 + (-83)$.

В естественной двоично-десятичной форме записи $X - Y = (0110\ 0111) - (1000\ 0011) = (0110\ 0111) + (-1000\ 0011)$. $[Y]_{\text{доп(с избытком 6)}}$ равно $0111\ 1101$, тогда операция вычитания реализуется следующим образом :

$0110\ 0111\ X + 0111\ 1101\ [Y]_{\text{доп(с избытком 6)}}$

$1110 \leftarrow 1101\ [X+Y]_{\text{доп.}}$ (с избытком 6)

Перенос из стареющей тетрады отсутствует, + следовательно сумма отрицательная.

1010 0000 коррекция

1000 0100 [X-Y]доп.(без избытка 6 в тетрадах).

Перевод суммы в прямой код.

0111 1100 [X-Y]пр.(с избытком 6 в каждой тетраде)

+1010 1010 коррекция

0001 0110 X – Y = -16 (истинное значение суммы).

В данном примере отсутствие переноса из старшей тетрады при сложении + X и дополнения для – Y указывают на то, что результат получился отрицательный в виде дополнения. Для определения истинного значения результата следует от полученного дополнения для [X-Y]доп взять новое дополнение, т.е. получить прямой код [X-Y]пр с избытком шесть, и из него потетрадно вычесть избыток 6, т.е. потетрадно прибавить 1010.

16) Операция умножения чисел в D-кодах.

Суть процесса состоит в том, что множемое, сдвинутое соответствующим образом относительно разрядной сетки, суммируется с суммой частичных произведений столько раз, сколько единиц содержится в очередной десятичной цифре множителя, на которую производится умножение. После этого сумма частичных произведений сдвигается вправо на один десятичный разряд и суммируется с множимым столько раз, сколько единиц содержится в очередном разряде множителя и т.д.

17) Операция деления чисел в D-кодах

Устройство содержит 2 регистра, Σ и счетчик. В P1- нах-ся частное, в P2- делитель, в суммароп зан-ся делимое и остаток.

P1(C) P2(B) $\Sigma(A)$

Правило деления. В первом такте каждого цикла сод-ся R1,(где нах-ся результат), сдвигается на одну тетраду в сторону старшего разряда, содер-ся R2 на одну тетраду в сторону мл. разряда. Во 2 фазе вып-ся ариф-ая операция. В нечетных циклах вып-ся вычитание сум-ра из R2. После каждого вычитания, если Σ не меняет знак, счетчик наращивается. Если меняет, то содержимое счетчика заносится в R1. В четных циклах вып-ся сложение Σ и R2. Начальное состояние ст=9. После каждой опер-ии, если знак не меняется, счетчик декрементируется. Если меняются, содер-ое заносится в R1 и переход в след. Цикл.

18. Бинарные отношения, способы задания бинарных отношений.

Пусть A и B два конечных множества. Декартовым произведением множеств A и B называют множество $A \times B$, состоящее из всех упорядоченных пар, где $a \in A, b \in B$.

Бинарным отношением между элементами множества A и B называется любое подмножество R множества $A \times B$, то есть $R \subset A \times B$.

По определению, бинарным отношением называется множество пар. Если R — бинарное отношение (т.е. множество пар), то говорят, что параметры x и y связаны бинарным отношением R , если пара (x, y) является элементом R , т.е. $(x, y) \in R$.

Способы задания бинарных отношений

1) Перечислить все пары, связанные между собой отношением.

2) Указать общие свойства, характеризующие данное отношение. Это наиболее общий способ, позволяющий задать практически любые отношения.

3) Графический способ, или задание отношения с помощью графа. В этом случае элементы множеств X и Y обозначаются точками, а элементы, связанные отношениями, соединяются направленными стрелками

4) Матричный способ. При этом отношение описывается матрицей, количество столбцов которой соответствует количеству элементов множества X , а строк – Y . Элемент матрицы, находящийся на пересечении j столбца и i строки равен 1, если соответствующие элементы множеств X и Y связаны бинарным отношением, и 0 - в противном случае.

19) Свойства бинарных отношений

1. Рефлексивность

Отношение рефлексивно, если для любого x вып. $xAx \text{ и } x \in A$

$1 \ 0 \ 0 \dots 0 \ E \leq A$

$E = 0 \ 1 \ 0 \dots 0$

.....

$0 \ 0 \ 0 \dots 1$

В реф. отношении по главной диагонали единицы. Отношение антирефлексивно, если $\langle x, x \rangle \notin A$. Отношение содержит 0 на гл. диагонали $E \cap A = \emptyset$; Отношение м.б. рефлексивным или антирефлексивным.

2. Симметричное отн-е, если каждой пары $\langle x, y \rangle$

$xAy \Rightarrow \langle y, x \rangle \in A$

Сим-ое отн-е симм-но относительно главной диагонали

$A \leq A^{-1}$

$A = A^{-1}$

Отношение наз. асимметричным $xAy \Rightarrow y \bar{A}x$; $\langle x, y \rangle \in A \Rightarrow \langle y, x \rangle \notin A$; $A \cap A^{-1} = \emptyset$;

Ассим-ое отн-е всегда антирефлексивно. $y \rightarrow x$; $xAx \Rightarrow x \bar{A}x$

+Антисимметричность. Если для каждой пары $\langle x, y \rangle$ вып. $xAy = yAx \Rightarrow x = y$ Отношение м.б. симм, ассим, или антисим. 3. Транзитивность. Если для каждого эл-та x, y, z вып. $xAy, yAz \Rightarrow xAz$; $A \leq A^2$

Транзитивное отн-е явл. Собственным транзитивным замыканием. $A = \bar{A}$

20) Толерантность, эквивалентность, отношения порядка

Толерантностью называют отношения, которые одновременно рефлексивны, симметричны, но не транзитивны, т.е. A - толерантность $\exists E \leq A \leq A^{-1} \wedge A^2 \not\leq A$

В качестве одного из примеров таких отношений можно привести отношение «иметь общее», которое еще называют отношением «сходства».

Если x имеет общее с y , то и y имеет общее с x , что говорит о свойстве симметричности данного отношения. Однако оно не транзитивно, поскольку из того, что x имеет сходство с y , а не с z , вовсе не следует сходство x и z (они уже могут не иметь ничего общего).

При последовательном переборе некоторой цепочки пар сходных элементов, образующей на графе «сходства» путь из вершины x в y , между которыми уже нет никакого сходства, происходит постепенное накопление свойств,

которыми обладает объект z и утрата свойств, присущих объекту x . Класс объектов, сходных с x , может пересекаться с классом объектов, сходных с z .

В область их пересечения попадут объекты, сходные одновременно и с x , и с z .

По аналогии с классами эквивалентности можно ввести понятие класса толерантности.

Если A — отношение толерантности, заданное на множестве X , то множество $C_i = \{x \in X: xAx_i\}$ будет классом толерантности,

образованным элементом $x_i \in X$ и содержащим все толерантные с ним элементы. Отметим основные отличия классов эквивалентности и толерантности.

- 1) Классы эквивалентности не пересекаются, а классы толерантности пересекаются.
- 2) Элементы класса эквивалентности попарно эквивалентны, а элементы класса толерантности, в общем и целом попарно нетолерантны.
- 3) Классы эквивалентности представляет собой монолит с совершенно равнозначными элементами, а класс толерантности имеет ядро и оболочку.

Ядро содержит один или более элементов. Элемент ядра является тем объектом x_i , который как раз и образует данный класс толерантности $C_i = \{x \in X: xAx_i\}$.

Элементы оболочки представляют довольно пеструю картину, некоторые их пары нетолерантны, а сама оболочка не имеет четких границ.

Эквивалентность. Эквивалентностью называют отношения, которые одновременно рефлексивны, симметричны и транзитивны, т.е. (A — эквивалентность) $\Leftrightarrow (E \subseteq A \subseteq A^{-1}) \wedge (A^2 \subseteq A)$.

Таким образом к эквивалентностям относятся такие отношения: (быть однополчанином, иметь тот же остаток при делении на 5 и т.д.

+Отношение порядка. Строгим порядком называют отношения, которые одновременно антирефлексивны и транзитивны,

т.е. (A — строгий порядок) $\Leftrightarrow (E \cap A = \emptyset) \wedge (A^2 \subseteq A)$.

Например отношение «меньше» антирефлексивно, т.к. условие $x < x$ не выполняется ни при каком значении x ,

и транзитивно следовательно, это строгий порядок. Другие примеры: больше, старше, быть потомком и т.д.

21) Транзитное замыкание

Транзитивное замыкание в [теории множеств](#) — это операция на [бинарных отношениях](#). Транзитивное замыкание бинарного отношения R на [множестве](#) X есть наименьшее [транзитивное отношение](#) на множестве X , включающее R .

Например, если X — это множество людей (и живых, и мёртвых), а R — отношение «является родителем», то транзитивное замыкание R — это отношение «является предком». Если X — это множество аэропортов, а xRy эквивалентно «существует рейс из x в y », и транзитивное замыкание R равно P , то xPy эквивалентно «можно долететь из x в y самолётом» (хотя иногда придётся лететь с пересадками)

22) Булевы (переключательные) функции. Способы задания булевых функций.

Булева функция от n переменных $(x_1, x_2, \dots, x_n)$ — это произвольное сюръективное отображение вида: $\{0, 1\}^n \rightarrow \{0, 1\}$. Булева функция двухзначных булевых переменных определена на множестве всех n -элементных кортежей или n -элементных последовательностей нулей и единиц (наборов переменных).

1) табличный метод

Функция задается в виде таблицы истинности, представляющей собой совокупность всех наборов переменных и соответствующих им значений функции.

x_1	x_2	x_3	y
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	1	0	0
1	1	1	1

2) аналитический метод

Функция задается в виде выражения, состоящего из обозначений всех входных переменных и необходимых логических операций между ними.

$$f_{\text{ша}} = \overline{x_1} \overline{x_2} x_3 + \overline{x_1} x_2 \overline{x_3} + x_1 \overline{x_2} \overline{x_3} + x_1 x_2 x_3$$

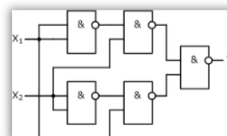
3) координатный метод

При этом способе дискретный автомат задается с помощью карты его состояний, которая известна как карта Карно или диаграмма Вейча. Карта Карно является компактной и очень удобной формой записи логической функции.

$x_1 x_2$		$x_3 x_4$			
		00	01	11	10
00		0	1	0	1
01		1	0	0	1
11		1	1	0	1
10		0	0	0	1

4) графический метод

Функция задается в виде схемы из логических элементов, которые соответствуют операциям, выполняемым над входными переменными. Схема может быть выполнена в соответствии с одним из общепринятых базисов.



23) Элементарные булевы функции двух переменных

Среди булевых функций особо выделяются так называемые **элементарные булевы функции**, посредством которых можно описать любую булеву функцию от любого числа переменных.

4. Конъюнкцией называется булева функция двух переменных, которая определяется следующей таблицей истинности:

x	0	0	1	1
y	0	1	0	1
$f(x, y)$	0	0	0	1

5. Дизъюнкцией называется булева функция двух переменных, которая определяется следующей таблицей истинности:

x	0	0	1	1
y	0	1	0	1
$f(x, y)$	0	1	1	1

6. Импликацией называется булева функция двух переменных, которая определяется следующей таблицей истинности:

x	0	0	1	1
y	0	1	0	1
$f(x, y)$	1	1	0	1

7. Эквивалентностью называется булева функция двух переменных, которая определяется следующей таблицей истинности:

x	0	0	1	1
y	0	1	0	1
$f(x, y)$	1	0	0	1

24) Тожества булевой алгебры. Элементарные преобразования

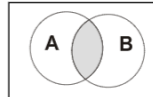
И — логическое умножение или конъюнкция

Таблица истинности операции И

A	B	$F = A \wedge B$
0	0	0
0	1	0
1	0	0
1	1	1

Диаграмма Эйлера — Венна

В алгебре множеств конъюнкции соответствует операция **пересечения** множеств.



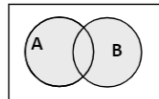
ИЛИ — логическое сложение или дизъюнкция

Таблица истинности операции ИЛИ

A	B	$F = A \vee B$
0	0	0
0	1	1

Диаграмма Эйлера-Венна

В алгебре множеств дизъюнкции соответствует операция **объединения** множеств.



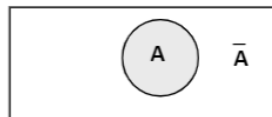
1	0	1
1	1	1

НЕ — логическое отрицание или инверсия

Таблица истинности операции НЕ

A	$F = \bar{A}$
0	1
1	0

Диаграмма Эйлера-Венна

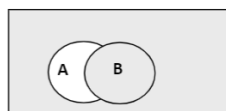


ЕСЛИ-ТО — логическое следование или импликация

Таблица истинности операции «импликация»

A	B	$F = A \rightarrow B$
0	0	1
0	1	1
1	0	0
1	1	1

Диаграмма Эйлера-Венна

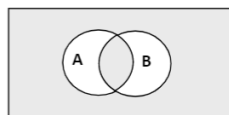


РАВНОСИЛЬНО — логическое равенство или эквиваленция

Таблица истинности операции «эквиваленция»

A	B	$F = A \leftrightarrow B$
0	0	1
0	1	0
1	0	0
1	1	1

Диаграмма Эйлера-Венна



25. Специальные классы булевых функций

Самодвойственной наз. булева функция, которая на противоположных наборах аргументов принимает противоположные значения, т.е. $f(x_1, x_2, \dots, x_n) = \bar{f}(\bar{x}_1, \bar{x}_2, \dots, \bar{x}_n)$ Монотонные функции наз. булева функция, если при возрастании наборов аргументов ее значения не убывают. Прим.

$(1,0,0,1) \not\subseteq (1,0,1,1)$ Данное отношение образует частичный порядок на множестве наборов аргументов.

Линейная функция. Функция $f(x_1, x_2, \dots, x_n)$ наз. линейной, если $a_0, a_1, \dots, a_n \in \{0, 1\}$ такие, что $f(x_1, x_2, \dots, x_n) = a_0 \oplus a_1 x_1 \oplus a_2 x_2 \oplus \dots \oplus a_n x_n$ если функция представлена полиномом Жегалкина. Операция конъюнкция ($a_1 x_1$ тождественно $a_1 \wedge x_1$) Функция сохраняющая константу Функция $f(x_1, x_2, \dots, x_n)$ наз. функцией, сохраняющей ноль, если $f(0, \dots, 0) = 0$. Функция $f(x_1, x_2, \dots, x_n)$ наз. функцией, сохраняющей 1, если $f(1, \dots, 1) = 1$. Теорема: В функционально-полной системе переключательных функций должна быть хотя бы одна функция, не являющаяся самодвойственной, монотонной, линейной, не сохраняющая 0 или 1. $\Rightarrow$ строгое формальное доказательство того, что любую булеву функцию можно выразить через определенный набор других функции, т.е. представить в некотором базисе. Опр. Система переключательных функций $\{f_1, f_2, \dots, f_k\}$ является полной в классе B и наз. базисом в том случае, если любая переключательная функция $f \in B$ может быть представлена посредством функций $f_1, f_2, \dots, f_k$ с использованием перенумерации аргументов или подстановки.

26. Днф.

Дизъюнктивной нормальной формой (ДНФ) называется дизъюнкция простых конъюнкций. Формула $(y_{11} \wedge y_{21} \wedge y_{n1}) \vee (y_{11} \wedge y_{21} \wedge y_{n1}) \vee \dots \vee V$ наз. СДНФ. Она представляет собой дизъюнкцию всех полных конъюнкций для всех вершин области истинности этой функции. СДНФ наз. совершенной, потому что каждое слагаемое в дизъюнкции включает все переменные. СДНФ $\Rightarrow$ для любой булевой функции, кроме $const = \text{ложь}$, Однако эту $const$, можно определить более простой формулой $0 = x_i \wedge \neg x_i$. Элементарная конъюнкция наз. полной если она является формулой для булевой функции (имеющий область истинности только из одной вершины гиперкуба)

27 Скнф.

Совершенной конъюнктивной нормальной формой (СКНФ) называется такая КНФ, у которой в каждую простую дизъюнкцию входят все переменные данного списка (либо сами, либо их отрицания), причем в одинаковом порядке. переход от ДНФ к КНФ: Алгоритм этого перехода следующий: ставим над ДНФ два отрицания и с помощью правил де Моргана (не трогая верхнее отрицание) приводим отрицание ДНФ снова к ДНФ. При этом приходится раскрывать скобки с использованием правила поглощения (или правила Блейка). Отрицание (верхнее) полученной ДНФ (снова по правилу де Моргана) сразу дает нам КНФ. КНФ можно получить и из первоначального выражения, если вынести $\neg$ за скобки.

28 Метод Квайна-Мак-Класки

Минимизация функции основывается на использовании правила поглощения, позволяющего вычислить все множество 1 - 2-, ..., n- кубов, образующих комплекс $K(f)$. Из этого комплекса выделяются кубы наибольшей размерности, покрывающие все множество вершин функции, определяя покрытие $Z(f)$ функции. Покрытие $Z(f)$ упрощается с целью получения минимального. В задачах минимизации булевых функций используется понятие простой импликанты. Некоторый куб $z \in K$ называется простой импликантой, если он не содержится ни в каком другом кубе комплекса K , т.е. не является гранью ника-кого другого куба из этого комплекса. Этапы минимизации: нахождение простых импликант. Все 0-кубы сравниваются попарно между собой на предмет образования 1-кубов. Если 0-кубы образуют 1-куб, они помечаются. Этап заканчивается, когда ни один куб более высокого порядка не может быть построен 2) Составление таблицы покрытий. Задача данного этапа - удалить все лишние простые импликанты. Определение существенных импликант. Если в каком-либо столбце таблицы покрытий имеется только одна метка, то соответствующая ей импликанта помечается как существенная. Данная

импликанта обязательно будет входить в минимальное покрытие, поскольку без нее невозможно покрыть все 0-кубы функции, в определяемое покрытие вносят все существенные импликанты, а из таблицы вычерчиваются соответствующие строки и столбцы, покрываемые данными импликантами.

4) Вычеркивание лишних столбцов. Если в остаточной таблице, полученной после выделения существенных импликант, имеются два столбца, имеющие метки в одинаковых строках, то один из них вычеркивается. 5. Вычеркивание лишних простых импликант. Если в остаточной таблице имеются строки, не имеющие ни одной метки, импликанты, соответствующие данным строкам, вычеркиваются. 6. Нахождение минимального покрытия.

29) Минимизация булевых функций методом Блейка. Примеры.

" Метод позволяет получать сокращенную ДНФ булевой функции f из ее произвольной ДНФ. Базируется на применении формулы обобщенного склеивания:

$$Ax \vee B/x = Ax \vee B/x \vee AB,$$

справедливость которой легко доказать. Действительно,

$$Ax = Ax \vee ABx; \quad B/x = B/x \vee AB/x.$$

Следовательно,

$$Ax \vee B/x = Ax \vee ABx \vee B/x \vee AB/x = Ax \vee B/x \vee AB.$$

В основу метода положено следующее утверждение: если в произвольной ДНФ булевой функции f произвести все возможные обобщенные склеивания, а затем выполнить все поглощения, то в результате получится сокращенная ДНФ функции f .

30) Не полностью определенные функции, минимизация не полностью определенных функций на картах Карно и методом Куайна-Мак-Класки.

Метод Куайна—Мак-Класки— табличный метод минимизации булевых функций, предложенный Уиллардом Куайном и усовершенствованный Эдвардом Мак-Класки. Представляет собой попытку избавиться от недостатков метода Куайна.

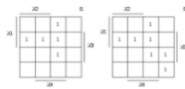
Специфика метода Куайна — Мак-Класки по сравнению с методом Куайна в сокращении количества попарных сравнений на предмет их склеивания. Сокращение достигается за счет исходного разбиения термов на группы с равным количеством единиц (нулей). Это позволяет исключить сравнения, заведомо не дающие склеивания.

Карта Карно— графический способ представления булевых функций с целью их удобной и наглядной ручной минимизации.

Является одним из эквивалентных способов описания или задания логических функций наряду с таблицей истинности или выражениями булевой алгебры. Преобразование карты Карно в таблицу истинности или в булеву формулу и обратно осуществляется элементарным алгоритмом.

F		$x_3 x_4$			
		00	01	11	10
$x_1 x_2$	00	1	0	0	1
	01	1	0	0	1
	11	0	1	1	0
	10	1	0	0	1

31) Минимизация систем переключательных функций



$K^0(n)$ – область истинности

$M^0(n)$ – область ложности

$$1. K^0_1(n) \leq K^0_2(n)$$

$$2. K^0_1(n) \leq M^0_2(n)$$

$$3. M^0_1(n) \leq M^0_2(n)$$

4. Произвольная

$$K^0_1(n) \leq K^0_2(n)$$

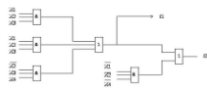
$$F_2 = F(f_1, x_1, x_2, \dots)$$

Функция выражается через другую функцию

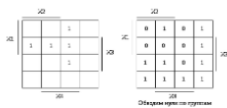
$$f_1 \min = x_1 x_2 x_3 \vee x_1 x_2 x_4 \vee x_2 x_3 x_4$$

$$f_2 \min = x_1 x_2 x_3 \vee x_1 x_2 x_4 \vee x_2 x_3 x_4 \vee x_1 x_2 x_4$$

$$f_2 \min = f_1 \min \vee x_1 x_2 x_4$$



31.2

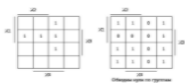


$$f_2 \min = x_1 x_2 x_3 \vee x_1 x_2 x_4 \vee x_1 x_2 x_4$$

$$f_2 \min = f_1 \min \vee x_1 x_2 x_4$$



3)



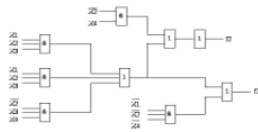
q

$$f_1 \min = (x_1 x_2 x_3 \vee x_1 x_2 x_4 \vee x_2 x_3 x_4)$$

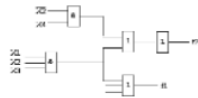
$$f_2 = q \vee x_2 x_4$$

$$f_{2 \min} = x_2 x_4 \vee x_1 x_2 x_3$$

31.3



4)



32) АЛГЕБРА ВЫСКАЗЫВАНИЙ

Высказывание – любые повествовательные предложения про которые можно сказать истинно оно или ложно.

Сигнатура алгебры множеств!

$$\{\neg, \wedge, \vee, \oplus, \Rightarrow, \neg\}$$

$$X \Rightarrow Y = \neg X \vee Y$$

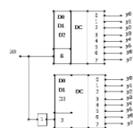
$$X \sim Y = XY = X \bar{Y} \sim Y \text{ - эквивалентность}$$

Высказывание – автоматическое если соответствует одному простому предложению (без знаков пунктуации, союзов)

Если дешифратор полный $m=2^N$

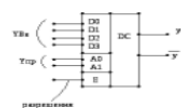
Где m – число входов

N – число выходов



$$X_4=0 - DC1$$

$$X_4=1 - DC2$$



если

$$y = E(A_0$$

$$A_1 D_0 \vee A_0 A_1 D_1 \vee A_0 A_1 D_2 \vee A_0 A_1 D_3) 32.2$$

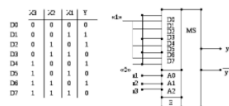
их двоичный код соответствует y_{BX}

$m = 2^n$ где n – число управляющих ($y_{упр}$)

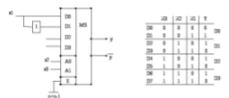
m – число входов (y_{BX})

$$y = f(x_1, x_2, x_3, \dots, x_n)$$

$$A_0 A_1 \dots A_{n-1} \{0, 1\} \rightarrow D_i$$



Если число элементов на 1 больше



Передается (n-1) – переменное значение оставшейся переменной сравнивается со значением функции, в независимости от того совпадают они или нет

$$\{0, 1, X, X\} \rightarrow D_1 D_1 = x_1 x_2$$

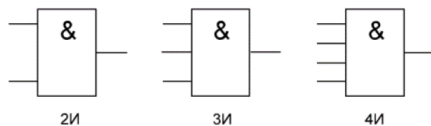
$$D_2 = x_1$$

$$D_3 = x_1$$

$$D_0 = x_1$$

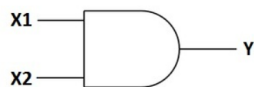
33) Реализация комбинационных схем в заданном базисе. Реализация комбинационных схем в классическом базисе («НЕ», «И», «ИЛИ»). Принципы реализации «по единицам» и «по нулям». Оценка сложности комбинационных схем.

Логический элемент «И» - конъюнкция, логическое умножение, AND



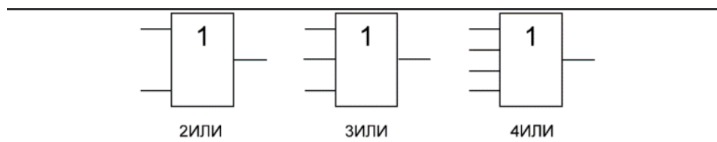
«И» - логический элемент, выполняющий над входными данными операцию конъюнкции или логического умножения. Данный элемент может иметь от 2 до 8 (наиболее распространены в производстве элементы «И» с 2, 3, 4 и 8 входами) входов и один выход.

Условные обозначения логических элементов «И» с разным количеством входов приведены на рисунке. В тексте логический элемент «И» с тем или иным числом входов обозначается как «2И», «4И» и т. д. - элемент «И» с двумя входами, с четырьмя входами и т. д.

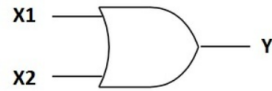


Вход X1	Вход X2	Выход Y
0	0	0
1	0	0
0	1	0
1	1	1

Таблица истинности для элемента 2И показывает, что на выходе элемента будет логическая единица лишь в том случае, если логические единицы будут одновременно на первом входе И на втором входе. В остальных трех возможных случаях на выходе будет ноль.



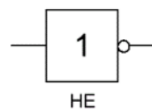
«ИЛИ» - логический элемент, выполняющий над входными данными операцию дизъюнкции или логического сложения. Он так же как и элемент «И» выпускается с двумя, тремя, четырьмя и т. д. входами и с одним выходом. Условные обозначения логических элементов «ИЛИ» с различным количеством входов показаны на рисунке. Обозначаются данные элементы так: 2ИЛИ, 3ИЛИ, 4ИЛИ и т. д.



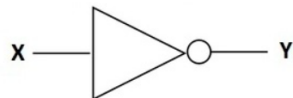
Вход X1	Вход X2	Выход Y
0	0	0
1	0	1
0	1	1
1	1	1

Таблица истинности для элемента «2ИЛИ» показывает, что для появления на выходе логической единицы, достаточно чтобы логическая единица была на первом входе ИЛИ на втором входе. Если логические единицы будут сразу на двух входах, на выходе также будет единица.

На западных схемах значок элемента «ИЛИ» имеет закругление на входе и закругление с заострением на выходе. На отечественных схемах — прямоугольник с символом «1».



«НЕ» - логический элемент, выполняющий над входными данными операцию логического отрицания. Данный элемент, имеющий один выход и только один вход, называют еще инвертором, поскольку он на самом деле инвертирует (обращает) входной сигнал. На рисунке приведено условное обозначение логического элемента «НЕ».



Вход X	Выход Y
0	1
1	0

Таблица истинности для инвертора показывает, что высокий потенциал на входе даёт низкий потенциал на выходе и наоборот.

На западных схемах значок элемента «НЕ» имеет форму треугольника с кружочком на выходе. На отечественных схемах — прямоугольник с символом «1», с кружком на выходе.

34. Реализация комбинационных схем в базисе Жегалкина («И», «ИСКЛ. ИЛИ», «1»).

Полином Жегалкина — многочлен над полем Z_2 , то есть полином с коэффициентами вида 0 и 1, где в качестве произведения берётся конъюнкция, а в качестве сложения — исключающее или.

Теорема Жегалкина — утверждение о существовании и единственности представления всякой булевой функции в виде полинома Жегалкина^[1].

Полином Жегалкина представляет собой сумму по модулю два попарно различных произведений неинвертированных переменных, где ни в одном произведении ни одна переменная не встречается больше одного раза, а также (если необходимо) константы 1^[1]. Формально полином Жегалкина можно представить в виде

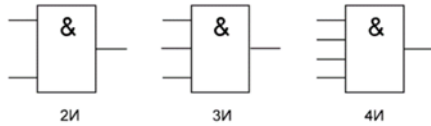
$$P(X_1, \dots, X_n) = a \oplus a_1 X_1 \oplus a_2 X_2 \oplus \dots \oplus a_n X_n \oplus a_{12} X_1 X_2 \oplus a_{13} X_1 X_3 \oplus \dots \oplus a_{1\dots n} X_1 \dots X_n,$$

$$a, \dots, a_{1\dots n} \in \{0, 1\},$$

или в более формализованном виде как

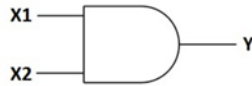
$$P = a \oplus \bigoplus_{\substack{1 \leq i_1 < \dots < i_k \leq n \\ k \in \overline{1, n}}} a_{i_1 \dots i_k} \wedge x_{i_1} \wedge \dots \wedge x_{i_k}, \quad a, a_{i_1 \dots i_k} \in \{0, 1\}.$$

Логический элемент «И» - конъюнкция, логическое умножение, AND



«И» - логический элемент, выполняющий над входными данными операцию конъюнкции или логического умножения. Данный элемент может иметь от 2 до 8 (наиболее распространены в производстве элементы «И» с 2, 3, 4 и 8 входами) входов и один выход.

Условные обозначения логических элементов «И» с разным количеством входов приведены на рисунке. В тексте логический элемент «И» с тем или иным числом входов обозначается как «2И», «4И» и т. д. - элемент «И» с двумя входами, с четырьмя входами и т. д.

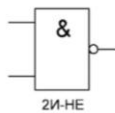


Вход X1	Вход X2	Выход Y
0	0	0
1	0	0
0	1	0
1	1	1

Таблица истинности для элемента 2И показывает, что на выходе элемента будет логическая единица лишь в том случае, если логические единицы будут одновременно на первом входе И на втором входе. В остальных трех возможных случаях на выходе будет ноль.

Исключающее «или» — булева функция, а также логическая и битовая операция, в случае двух переменных результат выполнения операции истинен тогда и только тогда, когда один из аргументов истинен, а другой — ложен. Для функции трёх (тернарное сложение по модулю 2) и более переменных — результат выполнения операции будет истинным только тогда, когда количество аргументов, равных 1, составляющих текущий набор, — нечётное. Такая операция естественным образом возникает в кольце вычетов по модулю 2, откуда и происходит название операции.

35.Реализация комбинационных схем в базисах «И-НЕ», «2И-НЕ», оценка сложности.



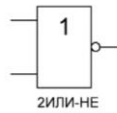
«И-НЕ» - логический элемент, выполняющий над входными данными операцию логического сложения, и затем операцию логического отрицания, результат подается на выход. Другими словами, это в принципе элемент «И», дополненный элементом «НЕ». На рисунке приведено условное обозначение логического элемента «2И-НЕ».



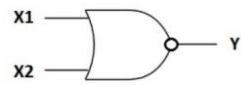
Вход X1	Вход X2	Выход Y
0	0	1
1	0	1
0	1	1
1	1	0

Таблица истинности для элемента «И-НЕ» противоположна таблице для элемента «И». Вместо трех нулей и единицы — три единицы и ноль. Элемент «И-НЕ» называют еще «элемент Шеффера» в честь математика Генри Мориса Шеффера, впервые отметившего значимость этой логической операции в 1913 году. Обозначается как «И», только с кружочком на выходе.

36.Реализация комбинационных схем в базисах «ИЛИ-НЕ», «2ИЛИ-НЕ», оценка сложности.



«ИЛИ-НЕ» - логический элемент, выполняющий над входными данными операцию логического сложения, и затем операцию логического отрицания, результат подается на выход. Иначе говоря, это элемент «ИЛИ», дополненный элементом «НЕ» - инвертором. На рисунке приведено условное обозначение логического элемента «2ИЛИ-НЕ».



Вход X1	Вход X2	Выход Y
0	0	1
1	0	0
0	1	0
1	1	0

Таблица истинности для элемента «ИЛИ-НЕ» противоположна таблице для элемента «ИЛИ». Высокий потенциал на выходе получается лишь в одном случае - на оба входа подаются одновременно низкие потенциалы. Обозначается как «ИЛИ», только с кружочком на выходе, обозначающим инверсию.

37.Реализация комбинационных схем на дешифраторах. (? Если успеем)

38.Реализация комбинационных схем на мультиплексорах. (? Если успеем)